



File Processing



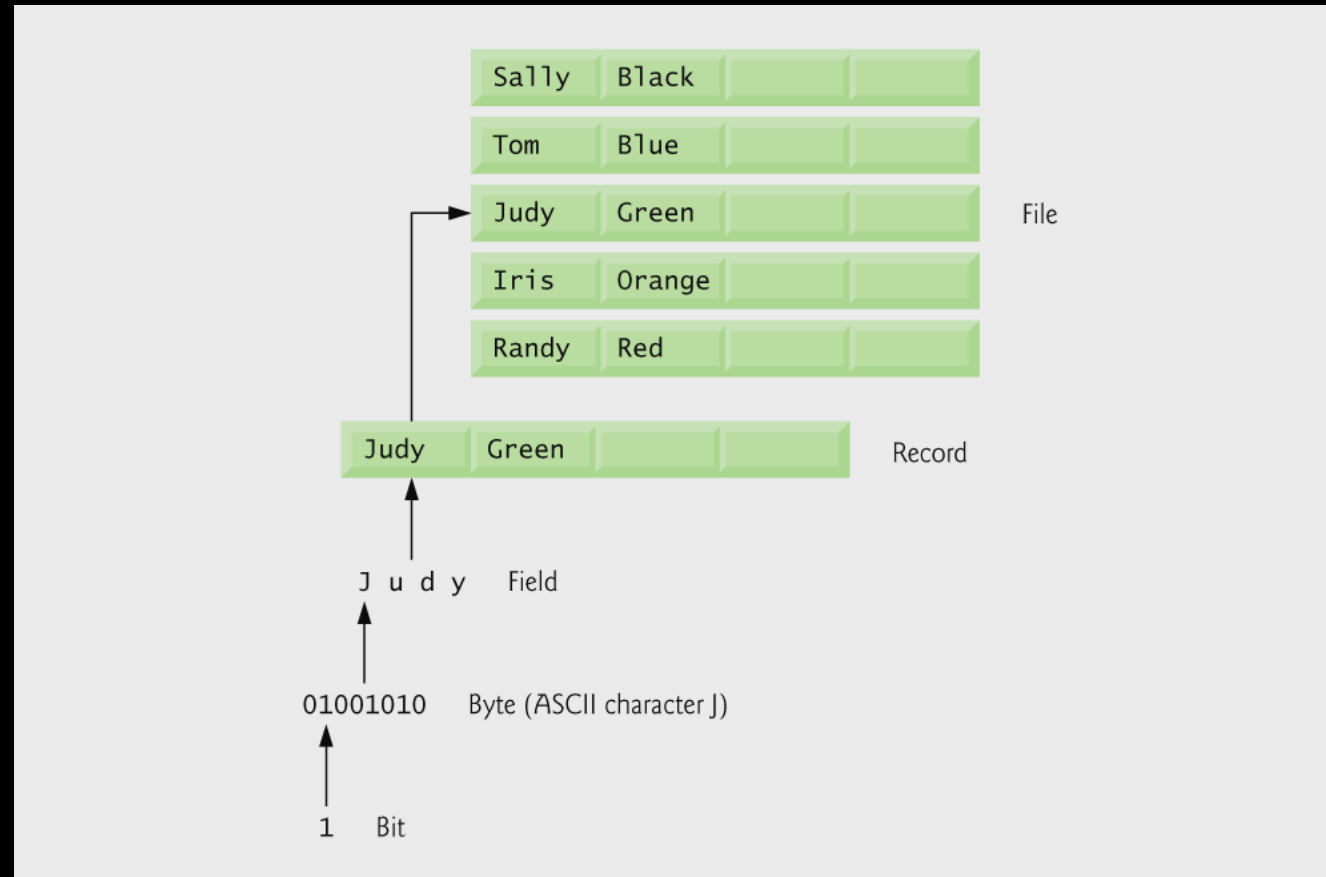
Introduction

- Data files
 - Can be created, updated, and processed by C programs
 - Are used for permanent storage of large amounts of data
 - Storage of data in variables and arrays is only temporary

Data Hierarchy

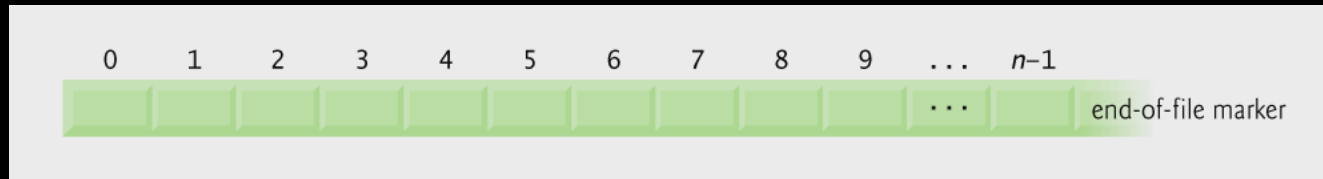
- Data Hierarchy:
 - Bit – smallest data item
 - Either value of 0 or 1
 - Byte – 8 bits
 - Used to store a character
 - Decimal digits, letters, and special symbols
 - Field – group of characters conveying meaning
 - Example: your name
 - Record – group of related fields
 - Represented by a struct
 - Example: In a payroll system, a **record** for a particular employee includes **his/her identification number, name, address**, etc.
 - File – group of related records
 - Example: a payroll file
 - Database – the group of related files

Data hierarchy



Files and Streams

- C views each file as a sequential stream of bytes
 - File ends with the *end-of-file marker (EOF)*
 - Or, file ends at a specified byte



- When a file is opened, a stream is associated with the file:
 - Provide communication channel between files and programs
 - Opening a file returns a pointer to a FILE structure
 - Example file pointers:
 - `stdin` - standard input (keyboard)
 - `stdout` - standard output (screen)
 - `stderr` - standard error (screen)

Files and Streams

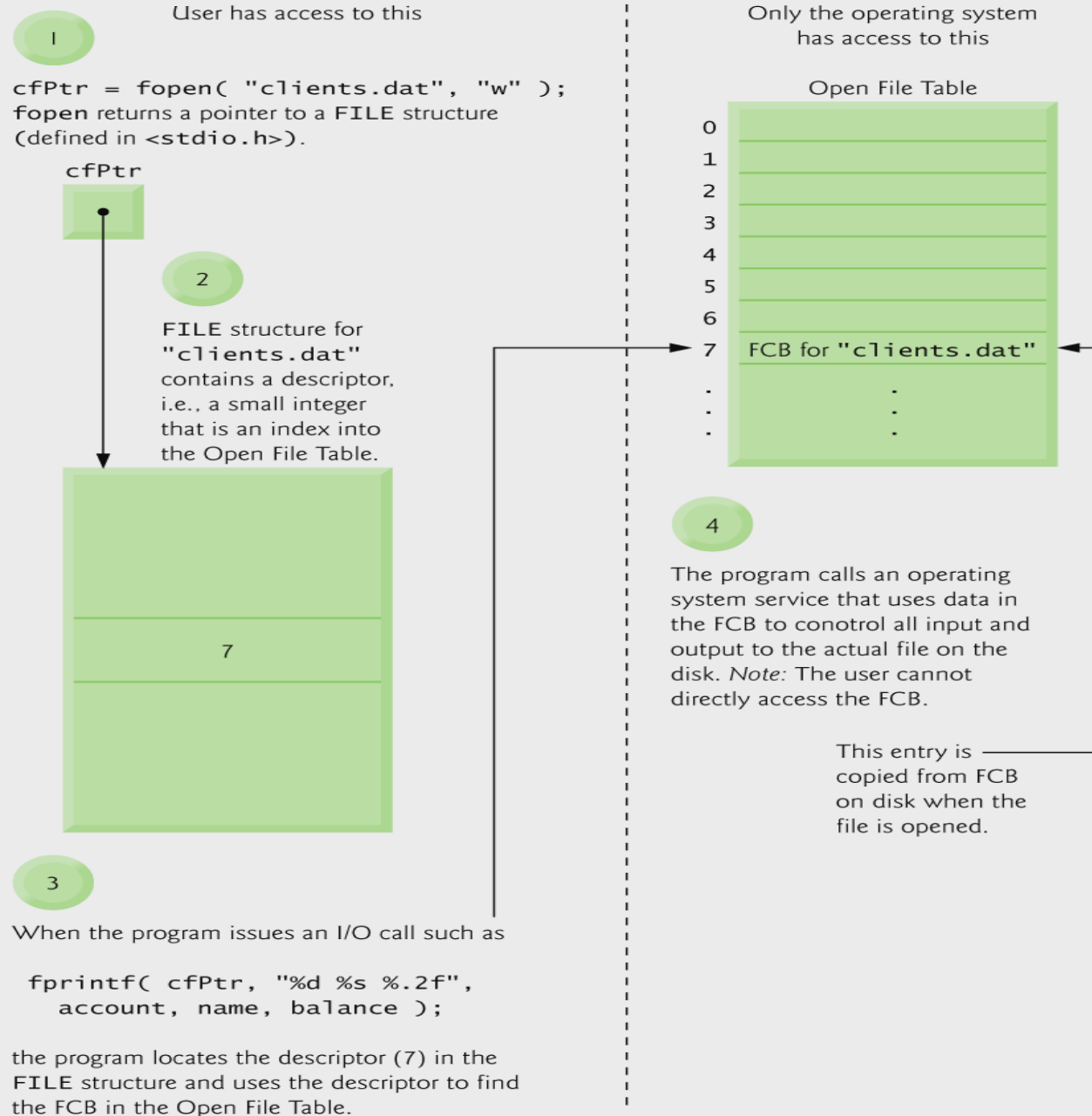
- FILE structure
 - File descriptor
 - A **file descriptor (FD)** is a **small integer (non-negative number)** used by the **operating system** to identify an open file or I/O resource.
 - Index into operating system array called the open file table
 - File Control Block (FCB)
 - system uses it to administer the file

```
struct _iobuf {  
    int _cnt;    // number of bytes left  
    char *_ptr;  // next character position  
    char *_base; // location of buffer  
    int _flag;   // mode of file (read/write/error)  
    int _file;   // file descriptor  
    int _charbuf; // one-char buffer  
    int _bufsiz; // size of buffer  
    char *_tmpfname; // temporary file name, if any  
};
```

File Descriptor Table (per process)

```
-----  
FD 0 → stdin (keyboard)  
FD 1 → stdout (screen)  
FD 2 → stderr (error output)  
FD 3 → myfile.txt (opened by fopen/fdopen)  
FD 4 → another file
```

Relationship between **FILE** pointers, **FILE** structures and FCBs



Files and Streams

- Read/Write functions in standard library
 - `fgetc`
 - Reads one character from a file
 - Takes a `FILE` pointer as an argument
 - `fgetc( stdin )` equivalent to `getchar()`
 - `fputc`
 - Writes one character to a file
 - Takes a `FILE` pointer and a character to write as an argument
 - `fputc( 'a', stdout )` equivalent to `putchar( 'a' )`
 - `fgets`
 - Reads a line from a file
 - `fputs`
 - Writes a line to a file
 - `fscanf` / `fprintf`
 - File processing equivalents of `scanf` and `printf`

Creating a Sequential-Access File

- C imposes no file structure
 - No notion of records in a file
 - Programmer must provide file structure
- Creating a File
 - `FILE *cfPtr;`
 - Creates a FILE pointer called cfPtr
 - `cfPtr = fopen("clients.dat", "w");`
 - Function `fopen` returns a FILE pointer to file specified
 - Takes two arguments – file to open and file open mode
 - If open fails, NULL returned

Creating a Sequential-Access File

- **fprintf**
 - Used to print to a file
 - Like **printf**, except first argument is a **FILE** pointer (pointer to the file you want to print in)
- **feof(*FILE*pointer)**
 - Returns true if end-of-file indicator (no more data to process) is set for the specified file
- **fclose(*FILE*pointer)**
 - Closes specified file
 - Performed automatically when program ends
 - Good practice to close files explicitly
- **Details**
 - Programs may process no files, one file, or many files
 - Each file must have a unique name and should have its own pointer

Creating a Sequential File: Example

```
1  /* Fig. 11.3: fig11_03.c
2     Create a sequential file */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int account;      /* account number */
8     char name[ 30 ]; /* account name */
9     double balance;   /* account balance */
10
11     FILE *cfPtr;      /* cfPtr = clients.dat file pointer */
12
13     /* fopen opens file. Exit program if unable to create file */
14     if ( ( cfPtr = fopen( "clients.dat", "w" ) ) == NULL ) {
15         printf( "File could not be opened\n" );
16     } /* end if */
17     else {
18         printf( "Enter the account, name, and balance.\n" );
19         printf( "Enter EOF to end input.\n" );
20         printf( "? " );
21         scanf( "%d%s%lf", &account, name, &balance );
22
```

FILE pointer definition creates new file pointer

fopen function opens a file; **w** argument means the file is opened for writing

Creating a Sequential File: Example

```
23  /* write account, name and balance into file with fprintf */
24  while ( !feof( stdin ) ) {
25      fprintf( cfPtr, "%d %s %.2f\n", account, name, balance );
26      printf( "? " );
27      scanf( "%d%s%f", &account, name, &balance );
28  } /* end while */
29
30  fclose( cfPtr ); /* fclose closes file */
31  } /* end else */
32
33  return 0; /* indicates successful termination */
34
35 } /* end main */
```

feof returns true when end of file is reached

fprintf writes a string to a file

fclose closes a file

```
Enter the account, name, and balance.
Enter EOF to end input.
? 100 Jones 24.98
? 200 Doe 345.67
? 300 White 0.00
? 400 Stone -42.16
? 500 Rich 224.62
? ^Z
```

End-of-file key combinations for various popular operating systems

Computer system	Key combination
UNIX systems	<i><return> <ctrl> d</i>
IBM PC and compatibles	<i><ctrl> z</i>
Macintosh	<i><ctrl> d</i>
Fig. 11.4 End-of-file key combinations for various popular computer systems.	

File opening modes

Mode	Description
r	Open an existing file for reading.
w	Create a file for writing. If the file already exists, discard the current contents.
a	Append; open or create a file for writing at the end of the file.
r+	Open an existing file for update (reading and writing).
w+	Create a file for update. If the file already exists, discard the current contents.
a+	Append: open or create a file for update; writing is done at the end of the file.
rb	Open an existing file for reading in binary mode.
wb	Create a file for writing in binary mode. If the file already exists, discard the current contents.
ab	Append; open or create a file for writing at the end of the file in binary mode.
rb+	Open an existing file for update (reading and writing) in binary mode.
wb+	Create a file for update in binary mode. If the file already exists, discard the current contents.
ab+	Append: open or create a file for update in binary mode; writing is done at the end of the file.

Reading Data from a Sequential-Access File

- Reading a sequential access file
 - Create a FILE pointer, link it to the file to read
`cfPtr = fopen( "clients.dat", "r" );`
 - Use `fscanf` to read from the file
 - Like `scanf`, except first argument is a FILE pointer
`fscanf( cfPtr, "%d%s%f", &account, name, &balance );`
 - Data read from beginning to end
 - File position pointer
 - Indicates number of next byte to be read / written
 - Not really a pointer, but an integer value (specifies byte location)
 - Also called **byte offset**
 - `rewind( cfPtr )`
 - Repositions file position pointer to beginning of file (byte 0)

Get current position : `ftell(FILE *fp)`

Reading a Sequential File: Example

```
1  /* Fig. 11.7: fig11_07.c
2     Reading and printing a sequential file */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int account;    /* account number */
8     char name[ 30 ]; /* account name */
9     double balance; /* account balance */
10
11     FILE *cfPtr;    /* cfPtr = clients.dat file pointer */
12
13     /* fopen opens file; exits program if file cannot be opened */
14     if ( ( cfPtr = fopen( "clients.dat", "r" ) ) == NULL ) {
15         printf( "File could not be opened\n" );
16     } /* end if */
17     else { /* read account, name and balance from file */
18         printf( "%-10s%-13s\n", "Account", "Name", "Balance" );
19         fscanf( cfPtr, "%d%s%lf", &account, name, &balance );
20     }
```

fopen function opens a file; **r** argument means the file is opened for reading

Reading a Sequential File: Example

```
21     /* while not end of file */
22     while ( !feof( cfPtr ) ) {
23         printf( "%-10d%-13s%7.2f\n", account, name, balance );
24         fscanf( cfPtr, "%d%s%f", &account, name, &balance );
25     } /* end while */
26
27     fclose( cfPtr ); /* fclose closes the file */
28 } /* end else */
29
30 return 0; /* indicates successful termination */
31
32 } /* end main */
```

fscanf function reads a string from a file

Account	Name	Balance
100	Jones	24.98
200	Doe	345.67
300	White	0.00
400	Stone	-42.16
500	Rich	224.62

Credit Inquiry: Example

```
1  /* Fig. 11.8: fig11_08.c
2     Credit inquiry program */
3  #include <stdio.h>
4
5  /* function main begins program execution */
6  int main( void )
7  {
8     int request;      /* request number */
9     int account;      /* account number */
10    double balance;   /* account balance */
11    char name[ 30 ];  /* account name */
12    FILE *cfPtr;      /* clients.dat file pointer */
13
14    /* fopen opens the file; exits program if file cannot be opened */
15    if ( ( cfPtr = fopen( "clients.dat", "r" ) ) == NULL ) {
16        printf( "File could not be opened\n" );
17    } /* end if */
18    else {
19
20        /* display request options */
21        printf( "Enter request\n"
22            " 1 - List accounts with zero balances\n"
23            " 2 - List accounts with credit balances\n"
24            " 3 - List accounts with debit balances\n"
25            " 4 - End of run\n? " );
26        scanf( "%d", &request );
27
```

Credit Inquiry: Example

```
28      /* process user's request */
29      while ( request != 4 ) {
30
31          /* read account, name and balance from file */
32          fscanf( cfPtr, "%d%s%f", &account, name, &balance );
33
34          switch ( request ) {
35
36              case 1:
37                  printf( "\nAccounts with zero balances:\n" );
38
39                  /* read file contents (until eof) */
40                  while ( !feof( cfPtr ) ) {
41
42                      if ( balance == 0 ) {
43                          printf( "%-10d%-13s%7.2f\n",
44                              account, name, balance );
45                      } /* end if */
46
47                      /* read account, name and balance from file */
48                      fscanf( cfPtr, "%d%s%f",
49                          &account, name, &balance );
50                  } /* end while */
51
52                  break;
53
```

Credit Inquiry: Example

```
54      case 2:
55          printf( "\nAccounts with credit balances:\n" );
56
57          /* read file contents (until eof) */
58          while ( !feof( cfPtr ) ) {
59
60              if ( balance < 0 ) {
61                  printf( "%-10d%-13s%7.2f\n",
62                      account, name, balance );
63              } /* end if */
64
65              /* read account, name and balance from file */
66              fscanf( cfPtr, "%d%s%1f",
67                  &account, name, &balance );
68          } /* end while */
69
70          break;
71
72      case 3:
73          printf( "\nAccounts with debit balances:\n" );
74
75          /* read file contents (until eof) */
76          while ( !feof( cfPtr ) ) {
77
78              if ( balance > 0 ) {
79                  printf( "%-10d%-13s%7.2f\n",
80                      account, name, balance );
81              } /* end if */
82
```

Credit Inquiry: Example

```
83         /* read account, name and balance from file */
84         fscanf( cfPtr, "%d%s%lf",
85             &account, name, &balance );
86     } /* end while */
87
88     break;
89
90 } /* end switch */
91
92 rewind( cfPtr ); /* return cfPtr to beginning of file */
93
94     printf( "\n? " );
95     scanf( "%d", &request );
96 } /* end while */
97
98     printf( "End of run.\n" );
99     fclose( cfPtr ); /* fclose closes the file */
100 } /* end else */
101
102 return 0; /* indicates successful termination */
103
104 } /* end main */
```

rewind function moves the file position pointer back to the beginning of the file

Credit Inquiry: Example

Enter request

- 1 - List accounts with zero balances
- 2 - List accounts with credit balances
- 3 - List accounts with debit balances
- 4 - End of run

? 1

Accounts with zero balances:

300	White	0.00
-----	-------	------

? 2

Accounts with credit balances:

400	Stone	-42.16
-----	-------	--------

? 3

Accounts with debit balances:

100	Jones	24.98
200	Doe	345.67
500	Rich	224.62

? 4

End of run.

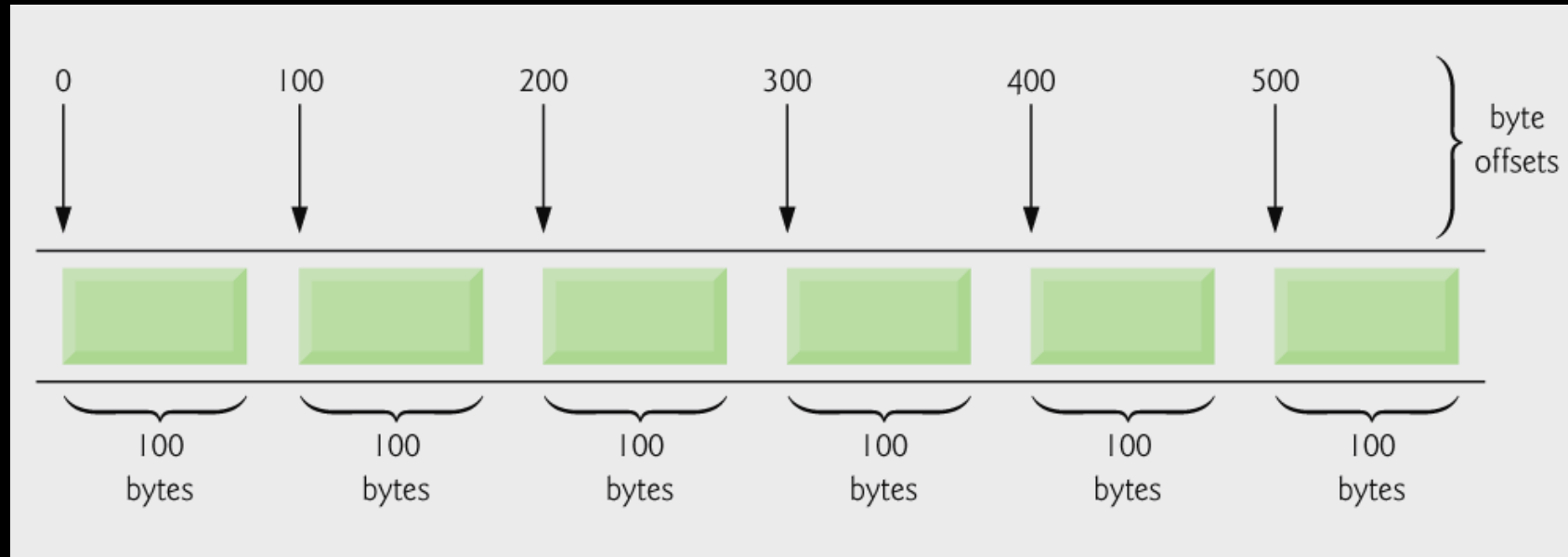
Updating a Sequential-Access File

- Sequential access file
 - Cannot be modified without the risk of destroying other data
 - Example:
 - 300 White 0.00
 - 300 Worthington 0.00
 - Characters beyond second 'o' will overwrite the beginning of the next sequential record
- Sequential access with fprintf and fscanf is not usually used to update records
- Requires processing every record in the file to update one record.
 - copying the records before 300 White 0.00 to a new file,
 - writing the new record,
 - copying the records after 300 White 0.00 to the new file, then
 - replacing the old file with the new one.

Random-Access Files

- Random access files
 - Access individual records without searching through other records
 - Instant access to records in a file
 - Data can be inserted without destroying other data
 - Data previously stored can be updated or deleted without overwriting
- Implemented using fixed length records
 - Sequential files do not have fixed length records

C's view of a random-access file



Creating a Random-Access File

- Data in random access files
 - Unformatted (stored as "raw bytes")
 - All data of the same type (`ints`, for example) uses the same amount of memory
 - All records of the same type have a fixed length. Hence, the exact location of a record relative to the beginning of the file can be calculated as a function of the record key.
 - Data not human readable

Method	What it Writes	Human Readable?
<code>fprintf(fPtr, "%d", number);</code>	Text (e.g., "300")	 Yes
<code>fwrite(&number, sizeof(int), 1, fPtr);</code>	Binary bytes (00000000 00000000 00000001 00101100)	 No

Creating a Random-Access File

- Unformatted I/O functions
 - `fwrite`
 - Transfer bytes from a location in memory to a file
 - `fread`
 - Transfer bytes from a file to a location in memory
 - Example:

```
fwrite( &number, sizeof( int ), 1, myPtr );
```

 - `&number` – Location to transfer bytes from
 - `sizeof( int )` – Number of bytes to transfer
 - `1` – For arrays, number of elements to transfer
 - In this case, "one element" of an array is being transferred
 - `myPtr` – File to transfer to or from

Creating a Random-Access File

- Writing structs

- `fwrite( &myObject, sizeof (struct myStruct), 1, myPtr );`

- `sizeof` – returns size in bytes of object in parentheses

- To write several array elements

- Pointer to array as first argument
 - Number of elements to write as third argument

Creating a Random-Access File: Example

```
1  /* Fig. 11.11: fig11_11.c
2     Creating a random-access file sequentially */
3  #include <stdio.h>
4
5  /* clientData structure definition */
6  struct clientData {
7     int acctNum; /* account number */
8     char lastName[ 15 ]; /* account last name */
9     char firstName[ 10 ]; /* account first name */
10    double balance; /* account balance */
11 }; /* end structure clientData */
12
13 int main( void )
14 {
15     int i; /* counter used to count from 1-100 */
16
17     /* create clientData with default information */
18     struct clientData blankClient = { 0, "", "", 0.0 };
19
```

Creating a Random-Access File: Example

```
20 FILE *cfPtr; /* credit.dat file pointer */
21
22 /* fopen opens the file; exits if file cannot be opened */
23 if ( ( cfPtr = fopen( "credit.dat", "wb" ) ) == NULL ) {
24     printf( "File could not be opened.\n" );
25 } /* end if */
26 else {
27
28     /* output 100 blank records to file */
29     for ( i = 1; i <= 100; i++ ) {
30         fwrite( &blankClient, sizeof( struct clientData ), 1, cfPtr );
31     } /* end for */
32
33     fclose ( cfPtr ); /* fclose closes the file */
34 } /* end else */
35
36 return 0; /* indicates successful termination */
37
38 } /* end main */
```

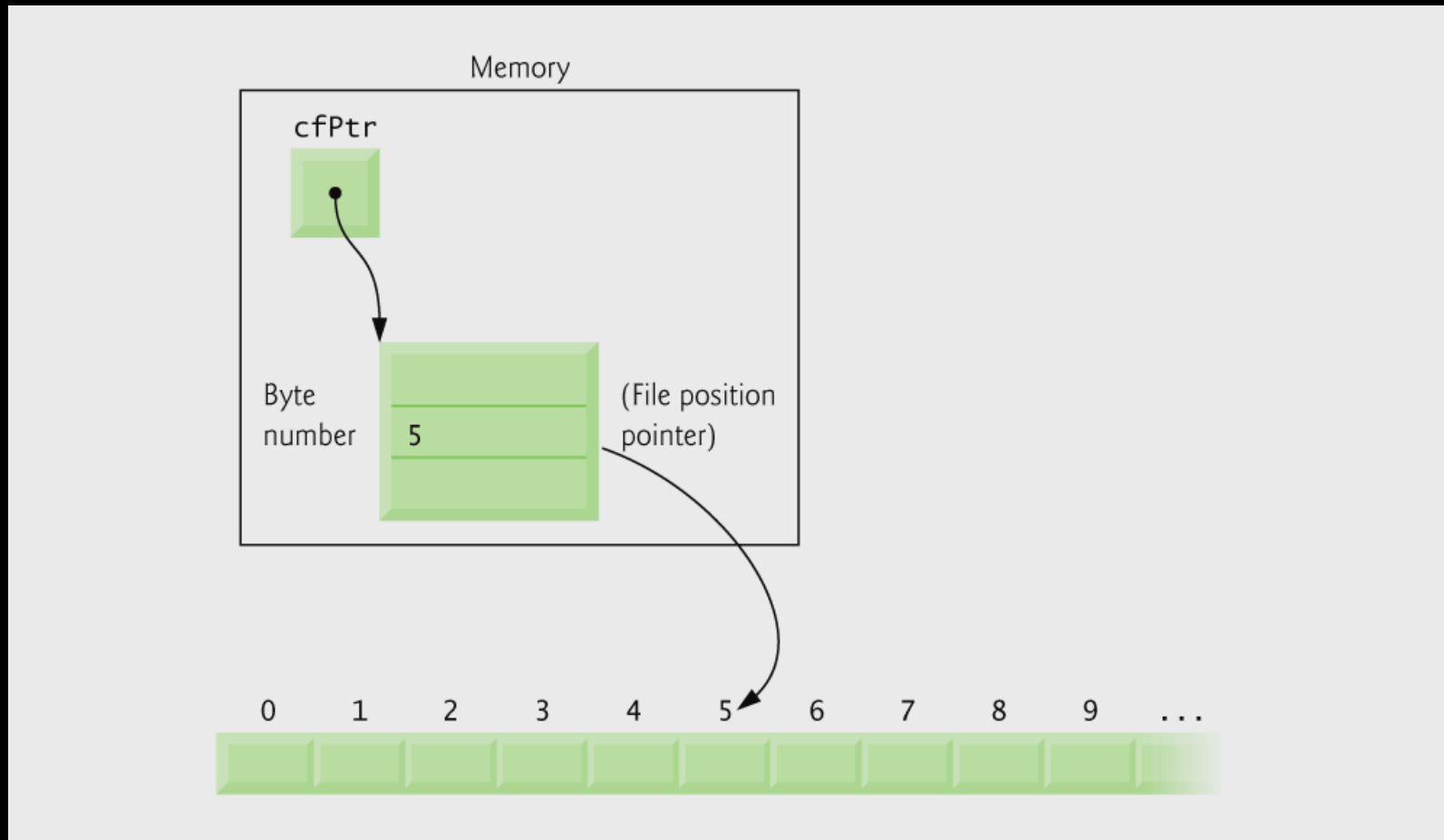
fopen function opens a file; **wb** argument means the file is opened for writing in binary mode

fwrite transfers bytes into a random-access file

Writing Data Randomly to a Random-Access File

- `fseek`
 - Sets file position pointer to a specific position
 - `fseek( pointer, offset, symbolic_constant ) ;`
 - *pointer* – pointer to file
 - *offset* – file position pointer (0 is first location)
 - *symbolic_constant* – specifies where in file we are reading from
 - `SEEK_SET` – seek starts at beginning of file
 - `SEEK_CUR` – seek starts at current location in file
 - `SEEK_END` – seek starts at end of file
 - Function `fseek` returns a **nonzero value** if the seek operation cannot be performed.

File position pointer indicating an offset of 5 bytes from the beginning of the file



Writing to a Random-Access File: Example

```
1  /* Fig. 11.12: fig11_12.c
2     Writing to a random access file */
3  #include <stdio.h>
4
5  /* clientData structure definition */
6  struct clientData {
7     int acctNum;      /* account number */
8     char lastName[ 15 ]; /* account last name */
9     char firstName[ 10 ]; /* account first name */
10    double balance;    /* account balance */
11 }; /* end structure clientData */
12
13 int main( void )
14 {
15     FILE *cfPtr; /* credit.dat file pointer */
16
17     /* create clientData with default information */
18     struct clientData client = { 0, "", "", 0.0 };
19
20     /* fopen opens the file; exits if file cannot be opened */
21     if ( ( cfPtr = fopen( "credit.dat", "rb+" ) ) == NULL ) {
22         printf( "File could not be opened.\n" );
23     } /* end if */
24     else {
25
26         /* require user to specify account number */
27         printf( "Enter account number"
28             " ( 1 to 100, 0 to end input )\n? " );
29         scanf( "%d", &client.acctNum );
30
```

Writing to a Random-Access File: Example

```
31  /* user enters information, which is copied into file */
32  while ( client.acctNum != 0 ) {
33
34      /* user enters last name, first name and balance */
35      printf( "Enter lastname, firstname, balance\n? " );
36
37      /* set record lastName, firstName and balance value */
38      fscanf( stdin, "%s%s%lf", client.lastName,
39              client.firstName, &client.balance );
40
41      /* seek position in file to user-specified record */
42      fseek( cfPtr, ( client.acctNum - 1 ) *
43             sizeof( struct clientData ), SEEK_SET );
44
45      /* write user-specified information in file */
46      fwrite( &client, sizeof( struct clientData ), 1, cfPtr );
47
48      /* enable user to input another account number */
49      printf( "Enter account number\n? " );
50      scanf( "%d", &client.acctNum );
51  } /* end while */
52
53      fclose( cfPtr ); /* fclose closes the file */
54  } /* end else */
55
56  return 0; /* indicates successful termination */
57
58 } /* end main */
```

fseek searches for a specific location in the random-access file

Writing to a Random-Access File: Example

```
Enter account number ( 1 to 100, 0 to end input )
? 37
Enter lastname, firstname, balance
? Barker Doug 0.00
Enter account number
? 29
Enter lastname, firstname, balance
? Brown Nancy -24.54
Enter account number
? 96
Enter lastname, firstname, balance
? Stone Sam 34.98
Enter account number
? 88
Enter lastname, firstname, balance
? Smith Dave 258.34
Enter account number
? 33
Enter lastname, firstname, balance
? Dunn Stacey 314.33
Enter account number
? 0
```

Reading Data from a Random-Access File

- `fread`
 - Reads a specified number of bytes from a file into memory
`fread( &client, sizeof (struct clientData), 1, myPtr );`
 - Can read several fixed-size array elements
 - Provide pointer to array
 - Indicate number of elements to read
 - To read multiple elements, specify in third argument

Reading a Random-Access File: Example

```
1  /* Fig. 11.15: fig11_15.c
2     Reading a random access file sequentially */
3  #include <stdio.h>
4
5  /* clientData structure definition */
6  struct clientData {
7     int acctNum;      /* account number */
8     char lastName[ 15 ]; /* account last name */
9     char firstName[ 10 ]; /* account first name */
10    double balance;    /* account balance */
11 }; /* end structure clientData */
12
13 int main( void )
14 {
15     FILE *cfPtr; /* credit.dat file pointer */
16
17     /* create clientData with default information */
18     struct clientData client = { 0, "", "", 0.0 };
19
20     /* fopen opens the file; exits if file cannot be opened */
21     if ( ( cfPtr = fopen( "credit.dat", "rb" ) ) == NULL ) {
22         printf( "File could not be opened.\n" );
23     } /* end if */
```

Reading a Random-Access File: Example

```
24  else {
25      printf( "%-6s%-16s%-11s%10s\n", "Acct", "Last Name",
26          "First Name", "Balance" );
27
28      /* read all records from file (until eof) */
29      while ( !feof( cfPtr ) ) {
30          fread( &client, sizeof( struct clientData ), 1, cfPtr );
31
32          /* display record */
33          if ( client.acctNum != 0 ) {
34              printf( "%-6d%-16s%-11s%10.2f\n",
35                  client.acctNum, client.lastName,
36                  client.firstName, client.balance );
37          } /* end if */
38
39      } /* end while */
40
41      fclose( cfPtr ); /* fclose closes the file */
42  } /* end else */
43
44  return 0; /* indicates successful termination */
45
46 } /* end main */
```

fread reads bytes from a random-access file to a location in memory

Acct	Last Name	First Name	Balance
29	Brown	Nancy	-24.54
33	Dunn	Stacey	314.33
37	Barker	Doug	0.00
88	Smith	Dave	258.34
96	Stone	Sam	34.98

Case Study: Transaction- Processing System

```
22 int main(void) {
23     FILE *cfPtr = NULL; // accounts.dat file pointer
24
25     // fopen opens the file; exits if file cannot be opened
26     if ((cfPtr = fopen("accounts.dat", "rb+")) == NULL) {
27         puts("File could not be opened.");
28     }
29     else {
30         int choice = 0; // user
31
32         // enable user to specify action
33         while ((choice = enterChoice()) != 5) {
34             switch (choice) {
35                 case 1: // create text file from record file
36                     textFile(cfPtr);
37                     break;
38                 case 2: // update record
39                     updateRecord(cfPtr);
40                     break;
41                 case 3: // create record
42                     newRecord(cfPtr);
43                     break;
44                 case 4: // delete existing record
45                     deleteRecord(cfPtr);
46                     break;
47                 default: // display message for invalid choice
48                     puts("Incorrect choice");
49                     break;
50             }
51         }
52
53         fclose(cfPtr); // fclose closes the file
54     }
55 }
```

Transaction- Processing System : Update Record

```
88 // update balance in record
89 void updateRecord(FILE *fPtr) {
90     // obtain number of account to update
91     printf("%s", "Enter account to update (1 - 100): ");
92     int account = 0; // account number
93     scanf("%d", &account);
94
95     // move file pointer to correct record in file
96     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
97
98     // read record from file
99     struct clientData client = {0, "", "", 0.0};
100    fread(&client, sizeof(struct clientData), 1, fPtr);
101
102    // display error if account does not exist
103    if (client.account == 0) {
104        printf("Account #%d has no information.\n", account);
105    }
106    else { // update record
107        printf("%-6d%-16s%-11s%10.2f\n\n", client.account, client.lastName,
108            client.firstName, client.balance);
109
110        // request transaction amount from user
111        printf("%s", "Enter charge (+) or payment (-): ");
112        double transaction = 0.0; // transaction amount
113        scanf("%lf", &transaction);
114        client.balance += transaction; // update record balance
115
116        printf("%-6d%-16s%-11s%10.2f\n", client.account, client.lastName,
117            client.firstName, client.balance);
118
119        // move file pointer to correct record in file
120        fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
121
122        // write updated record over old record in file
123        fwrite(&client, sizeof(struct clientData), 1, fPtr);
124    }
125 }
```


Transaction- Processing System : New Record

```
127 // create and insert record
128 void newRecord(FILE *fPtr) {
129     // obtain number of account to create
130     printf("%s", "Enter new account number (1 - 100): ");
131     int account = 0; // account number
132     scanf("%d", &account);
133
134     // move file pointer to correct record in file
135     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
136
137     // read record from file
138     struct clientData client = {0, "", "", 0.0};
139     fread(&client, sizeof(struct clientData), 1, fPtr);
140
141     // display error if account already exists
142     if (client.account != 0) {
143         printf("Account #%d already contains information.\n",
144             client.account);
145     }
146     else { // create record
147         // user enters last name, first name and balance
148         printf("%s", "Enter lastname, firstname, balance\n? ");
149         scanf("%14s%9s%lf", &client.lastName, &client.firstName,
150             &client.balance);
151
152         client.account = account;
153
154         // move file pointer to correct record in file
155         fseek(fPtr, (client.account - 1) * sizeof(struct clientData),
156             SEEK_SET);
157
158         // insert record in file
159         fwrite(&client, sizeof(struct clientData), 1, fPtr);
160     }
161 }
```

Transaction- Processing System : Delete Record

```
163 // delete an existing record
164 void deleteRecord(FILE *fPtr) {
165     // obtain number of account to delete
166     printf("%s", "Enter account number to delete (1 - 100): ");
167     int account = 0; // account number
168     scanf("%d", &account);
169
170     // move file pointer to correct record in file
171     fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
172
173     // read record from file
174     struct clientData client = {0, "", "", 0.0};
175     fread(&client, sizeof(struct clientData), 1, fPtr);
176
177     // display error if record does not exist
178     if (client.account == 0) {
179         printf("Account %d does not exist.\n", account);
180     }
181     else { // delete record
182         // move file pointer to correct record in file
183         fseek(fPtr, (account - 1) * sizeof(struct clientData), SEEK_SET);
184
185         struct clientData blankClient = {0, "", "", 0}; // blank client
186
187         // replace existing record with blank record
188         fwrite(&blankClient, sizeof(struct clientData), 1, fPtr);
189     }
190 }
```

